다항식 이론

개요

다항식 이론은 zkSNARK 프로토콜의 핵심 구성 요소입니다. 특히 다항식 보간, 평가, 그리고 sumcheck 프로토콜에서 중요한 역할을 합니다.

다항식 보간과 평가

라그랑주 보간법

기본 원리

n+1개의 점 $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$이 주어졌을 때, 이들을 지나는 유일한 n차 다항식을 구성합니다.

라그랑주 기저 다항식

```
L_i(x) = Π(j≠i) (x - x_j) / (x_i - x_j)
```

보간 다항식

```
P(x) = Σ_i y_i · L_i(x)
```

구현

```
template<typename FieldT>
```

```
class LagrangeInterpolation {
private:
vector<FieldT> x_coords;
vector<FieldT> y_coords;
public:
LagrangeInterpolation(
const vector<FieldT>&  x_vals,
const vector<FieldT>&  y_vals
) : x_coords(x_vals), y_coords(y_vals) {}
// 특정 점에서의 다항식 값 계산
FieldT evaluate(const FieldT&  x) const {
FieldT result = FieldT::zero();
size_t n = x_coords.size();
for (size_t i = 0; i < n; i++) {
FieldT basis = FieldT::one();
// 라그랑주 기저 다항식 계산
for (size_t j = 0; j < n; j++) {
if (i != j) {
basis *= (x - x_coords[j]) *
(x_coords[i] - x_coords[j]).inverse();
}
}
result += y_coords[i] * basis;
```

```
}
return result;
}
// 다항식 계수 계산
vector<FieldT> get_coefficients() const {
size_t n = x_coords.size();
vector<FieldT> coeffs(n, FieldT::zero());
for (size_t i = 0; i < n; i++) {
// i번째 라그랑주 기저의 계수들
vector<FieldT> basis_coeffs =
compute_lagrange_basis_coeffs(i);
for (size_t j = 0; j < n; j++) {
coeffs[j] += y_coords[i] * basis_coeffs[j];
}
}
return coeffs;
}
};
```

최적화된 보간법

뉴턴 보간법

```
template<typename FieldT>
```

```
vector<FieldT> newton_interpolation(
const vector<FieldT>&  x_vals,
const vector<FieldT>&  y_vals
) {
size_t n = x_vals.size();
vector<vector<FieldT>> divided_diff(n, vector<FieldT>(n));
// 0차 차분 초기화
for (size_t i = 0; i < n; i++) {
divided_diff[i][0] = y_vals[i];
}
// 고차 차분 계산
for (size_t j = 1; j < n; j++) {
for (size_t i = 0; i < n - j; i++) {
divided_diff[i][j] =
(divided_diff[i+1][j-1] - divided_diff[i][j-1]) /
(x_vals[i+j] - x_vals[i]);
}
}
// 뉴턴 형식을 표준 형식으로 변환
return  convert_newton_to_standard(divided_diff, x_vals);
}
```

다변수 다항식

기본 개념

- **다변수 다항식**: $P(x_1, x_2, ..., x_k)$
- **전체 차수**: 각 항의 차수 중 최대값
- **부분 차수**: 특정 변수에 대한 차수

표현 방법

```
template<typename FieldT>
class MultivariatePolynomial {
private:
map<vector<size_t>, FieldT> terms; // 지수 벡터 → 계수
size_t num_variables;
public:
// 항 추가
void  add_term(const vector<size_t>&  exponents, const FieldT&  coeff) {
if (coeff != FieldT::zero()) {
terms[exponents] += coeff;
}
}
// 다항식 평가
FieldT evaluate(const vector<FieldT>&  point) const {
FieldT result = FieldT::zero();
```

```
for (const auto& term : terms) {
FieldT term_value = term.second;
for (size_t i = 0; i < num_variables; i++) {
term_value *= point[i].pow(term.first[i]);
}
result += term_value;
}
return result;
}
// 부분 평가 (일부 변수만 대입)
MultivariatePolynomial partial_evaluate(
size_t var_index,
const FieldT& value
) const {
MultivariatePolynomial result(num_variables - 1);
for (const auto& term : terms) {
vector<size_t> new_exponents = term.first;
FieldT new_coeff = term.second;
// var_index 변수에 value 대입
new_coeff *= value.pow(new_exponents[var_index]);
new_exponents.erase(new_exponents.begin() + var_index);
result.add_term(new_exponents, new_coeff);
}
```

```
return result;
}
};
```

Sumcheck 프로토콜

기본 아이디어

다변수 다항식 $P(x_1, ..., x_k)$에 대해 다음 합을 검증:

$$S = \Sigma_{x_1 \in \{0,1\}} \ldots \Sigma_{x_k \in \{0,1\}} P(x_1, \ldots, x_k)$$

프로토콜 구조

```
template<typename FieldT>
class SumcheckProtocol {
private:
MultivariatePolynomial<FieldT> polynomial;
size_t num_variables;
vector<FieldT> challenges;
public:
// 라운드 i에서의 단변수 다항식 생성
vector<FieldT> round_polynomial(size_t  round) const {
// 이전 라운드의 챌린지들로 부분 평가
```

```
MultivariatePolynomial<FieldT> partial_poly = polynomial;

for (size_t i = 0; i < round; i++) {

partial_poly = partial_poly.partial_evaluate(0, challenges[i]);

}

// 현재 변수에 대한 단변수 다항식 계산

vector<FieldT> univariate_coeffs;

for (size_t degree = 0; degree <= polynomial.degree(); degree++) {

FieldT coeff = FieldT::zero();

// 모든 가능한 할당에 대해 합계 계산

for (size_t assignment = 0;

assignment < (1 << (num_variables - round - 1));

assignment++) {

vector<FieldT> point =

construct_evaluation_point(round, degree, assignment);

coeff += partial_poly.evaluate(point);

}

univariate_coeffs.push_back(coeff);

}

return univariate_coeffs;

}

// 검증자의 챌린지 처리

void  process_challenge(const FieldT&  challenge) {

challenges.push_back(challenge);
```

```
}

// 최종 검증

bool verify_final_claim(const FieldT& claimed_value) const {

vector<FieldT> final_point = challenges;

return polynomial.evaluate(final_point) == claimed_value;

}

};
```

Sumcheck 최적화

메모리 효율적 구현

```
template<typename FieldT>

class OptimizedSumcheck {

private:

// 스트리밍 방식으로 다항식 처리

function<FieldT(const vector<FieldT>&)> evaluation_function;

public:

vector<FieldT> compute_round_polynomial(

size_t round,

const vector<FieldT>& partial_challenges

) const {

vector<FieldT> coeffs(degree + 1, FieldT::zero());

// 병렬 처리로 성능 향상
```

```
#pragma omp parallel for
for (size_t assignment = 0;
assignment < (1 << (num_variables - round - 1));
assignment++) {
for (size_t x = 0; x <= degree; x++) {
vector<FieldT> point =
construct_point(round, x, assignment, partial_challenges);
FieldT value = evaluation_function(point);
#pragma omp atomic
coeffs[x] += value;
}
}
return coeffs;
}
};
```

다항식 연산 최적화

호너의 방법 (Horner's Method)

```
template<typename FieldT>
FieldT horner_evaluate(
const vector<FieldT>& coeffs,
```

```
const FieldT& x
) {
if (coeffs.empty()) return FieldT::zero();
FieldT result = coeffs.back();
for (int i = coeffs.size() - 2; i >= 0; i--) {
result = result * x + coeffs[i];
}
return result;
}
```

배치 평가

```
template<typename FieldT>
vector<FieldT> batch_evaluate(
const vector<FieldT>& coeffs,
const vector<FieldT>& points
) {
vector<FieldT> results(points.size());
#pragma omp parallel for
for (size_t i = 0; i < points.size(); i++) {
results[i] = horner_evaluate(coeffs, points[i]);
}
return results;
```

```
}
```

다항식 나눗셈

```
template<typename FieldT>
pair<vector<FieldT>, vector<FieldT>> polynomial_division(
const vector<FieldT>&  dividend,
const vector<FieldT>&  divisor
) {
if (divisor.empty() || divisor.back() == FieldT::zero()) {
throw  invalid_argument("Invalid divisor");
}
vector<FieldT> quotient;
vector<FieldT> remainder = dividend;
FieldT leading_coeff_inv = divisor.back().inverse();
while (remainder.size() >= divisor.size() &&
remainder.back() != FieldT::zero()) {
FieldT coeff = remainder.back() * leading_coeff_inv;
quotient.insert(quotient.begin(), coeff);
// 나머지 업데이트
for (size_t i = 0; i < divisor.size(); i++) {
size_t pos = remainder.size() - divisor.size() + i;
remainder[pos] -= coeff * divisor[i];
```

```
}

remainder.pop_back();

}

return  make_pair(quotient, remainder);

}
```

다음 단계

다음 문서에서는 Reed-Solomon 코드를 통해 다항식 인코딩과 근접성 테스트를 살펴보겠습니다.

← 이전: 유한체 연산 | 다음: Reed-Solomon 코드 →